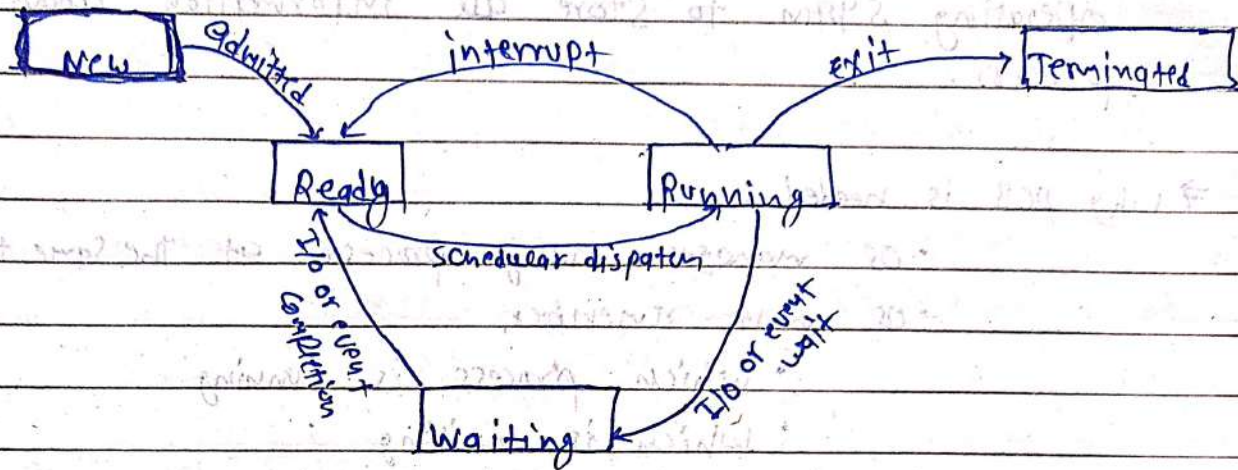


## \* Process States:



(i) ~~the~~ New:

- process is being created
- PCB is created and memory is allocated

(ii) Ready:

- process is in "main memory".
- Ready to run waiting for "cpu time".

(iii) Running:

- process is currently executing on cpu

(iv) Waiting:

- process is waiting for an event.
- example:
  - I/O completion
  - ~~the~~ user input

(v) Terminated:

- process has finished execution
- Resources are ~~the~~ released



$\log_2(2) = 1$   
 $\log_2(4) = 2$   
 $\log_2(8) = 3$   
 $\log_2(16) = 4$   
 $\log_2(32) = 5$   
 $\log_2(64) = 6$   
 $\log_2(128) = 7$   
 $\log_2(256) = 8$   
 $\log_2(512) = 9$   
 $\log_2(1024) = 10$   
 $\log_2(2048) = 11$   
 $\log_2(4096) = 12$   
 $\log_2(8192) = 13$   
 $\log_2(16384) = 14$   
 $\log_2(32768) = 15$   
 $\log_2(65536) = 16$   
 $\log_2(131072) = 17$   
 $\log_2(262144) = 18$   
 $\log_2(524288) = 19$   
 $\log_2(1048576) = 20$   
 $\log_2(2097152) = 21$   
 $\log_2(4194304) = 22$   
 $\log_2(8388608) = 23$   
 $\log_2(16777216) = 24$   
 $\log_2(33554432) = 25$   
 $\log_2(67108864) = 26$   
 $\log_2(134217728) = 27$   
 $\log_2(268435456) = 28$   
 $\log_2(536870912) = 29$   
 $\log_2(1073741824) = 30$   
 $\log_2(2147483648) = 31$   
 $\log_2(4294967296) = 32$   
 $\log_2(8589934592) = 33$   
 $\log_2(17179869184) = 34$   
 $\log_2(34359738368) = 35$   
 $\log_2(68719476736) = 36$   
 $\log_2(137438953472) = 37$   
 $\log_2(274877906944) = 38$   
 $\log_2(549755813888) = 39$   
 $\log_2(1099511627776) = 40$   
 $\log_2(2199023255552) = 41$   
 $\log_2(4398046511104) = 42$   
 $\log_2(8796093022208) = 43$   
 $\log_2(17592186044416) = 44$   
 $\log_2(35184372088832) = 45$   
 $\log_2(70368744177664) = 46$   
 $\log_2(140737488355328) = 47$   
 $\log_2(281474976710656) = 48$   
 $\log_2(562949953421312) = 49$   
 $\log_2(1125899906842624) = 50$   
 $\log_2(2251799813685248) = 51$   
 $\log_2(4503599627370496) = 52$   
 $\log_2(9007199254740992) = 53$   
 $\log_2(18014398509481984) = 54$   
 $\log_2(36028797018963968) = 55$   
 $\log_2(72057594037927936) = 56$   
 $\log_2(144115188075855872) = 57$   
 $\log_2(288230376151711744) = 58$   
 $\log_2(576460752303423488) = 59$   
 $\log_2(1152921504606846976) = 60$   
 $\log_2(2305843009213693952) = 61$   
 $\log_2(4611686018427387904) = 62$   
 $\log_2(9223372036854775808) = 63$   
 $\log_2(18446744073709551616) = 64$   
 $\log_2(36893488147419103232) = 65$   
 $\log_2(73786976294838206464) = 66$   
 $\log_2(147573952589676412928) = 67$   
 $\log_2(295147905179352825856) = 68$   
 $\log_2(590295810358705651712) = 69$   
 $\log_2(1180591620717411303424) = 70$   
 $\log_2(2361183241434822606848) = 71$   
 $\log_2(4722366482869645213696) = 72$   
 $\log_2(9444732965739290427392) = 73$   
 $\log_2(18889465931478580854784) = 74$   
 $\log_2(37778931862957161709568) = 75$   
 $\log_2(75557863725914323419136) = 76$   
 $\log_2(151115727451828646838272) = 77$   
 $\log_2(302231454903657293676544) = 78$   
 $\log_2(604462909807314587353088) = 79$   
 $\log_2(1208925819614629174706176) = 80$   
 $\log_2(2417851639229258349412352) = 81$   
 $\log_2(4835703278458516698824704) = 82$   
 $\log_2(9671406556917033397649408) = 83$   
 $\log_2(19342813113834066795298816) = 84$   
 $\log_2(38685626227668133590597632) = 85$   
 $\log_2(77371252455336267181195264) = 86$   
 $\log_2(154742504910672534362390528) = 87$   
 $\log_2(309485009821345068724781056) = 88$   
 $\log_2(618970019642690137449562112) = 89$   
 $\log_2(1237940039285380274899124224) = 90$   
 $\log_2(2475880078570760549798248448) = 91$   
 $\log_2(4951760157141521099596496896) = 92$   
 $\log_2(9903520314283042199192993792) = 93$   
 $\log_2(19807040628566084398385987584) = 94$   
 $\log_2(39614081257132168796771975168) = 95$   
 $\log_2(79228162514264337593543950336) = 96$   
 $\log_2(158456325028528675187087900672) = 97$   
 $\log_2(316912650057057350374175801344) = 98$   
 $\log_2(633825300114114700748351602688) = 99$   
 $\log_2(1267650600228229401496703205376) = 100$

\* Process Control block: A process Control block (PCB) is a data structure used by the operating system to store all information about a process.

⇒ Why PCB is needed:

- OS manages many processes at the same time
- OS must remember
  - Which process is running
  - Which is waiting
  - What resources a process is using.
- PCB helps OS track and control each process.

PCB Diagram:

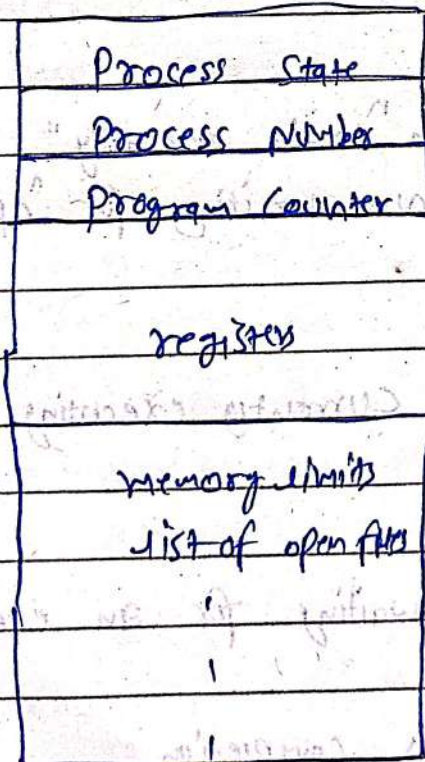


Figure: Process Control block.

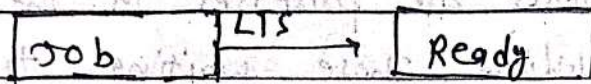


- **Process State :** It stores the current state of the process (new, ready, running, waiting, terminated).
- **Process number (PID) :** The PID in PCB stores the unique identification number of the process.
- **Program Counter :** Program Counter in PCB stores the address of the next instruction to be executed by the process.
- **Registers :** Registers in PCB stores all CPU register values required to restart process exactly from it was stopped.
- **Memory limits :** Memory limits in PCB store the information about the memory space allocated to the process, such as base (starting point of memory) and limit values (ending point of memory).
- **List of open files :** The list of open files in PCB stores information about all files currently opened by the process.



## ⇒ Types of Schedulers:

i) Long-term Scheduler (Job Scheduler): This scheduler selects a process from job queue (secondary memory) to ready queue (primary memory):



→ The long term scheduler Controls the "degree of multiprogramming". Meaning it manages the number of process can present in the ready queue to be executed.

• Works:

- Less frequently
- slower.

ii) Short-term Scheduler (CPU Scheduler): The short term Scheduler selects

process from ready queue and assign CPU to executed.

• purpose:

- Decides which process runs next
- improves CPU utilization

• works:

- very frequently
- very fast



17

\* Scheduling Queues: Scheduling queues are lists of processes waiting for different resources or execution stages in the operating system.

⇒ Types of Scheduling queues:

(i) Job queue: Stores all processes in the system, including those waiting to enter main memory.

(ii) Ready queue: Stores processes that are in main memory, and ready to execute, waiting for CPU.

(iii) Waiting queue: Stores processes waiting for I/O operations or events.

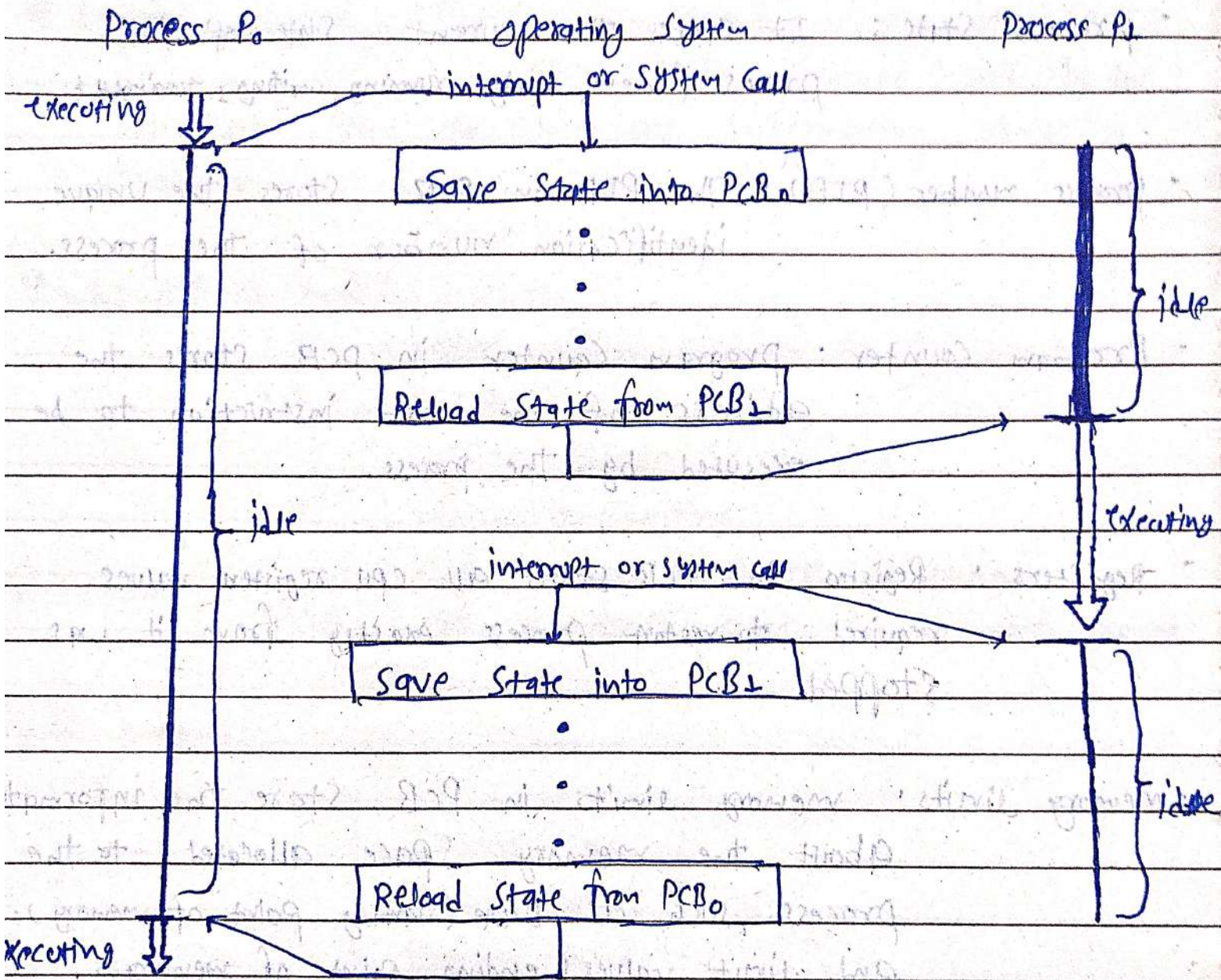
(iv) Device queue: Store processes waiting for a specific device (printer, disk, etc).

\* Schedulers: Schedulers are OS components that decide which process should be executed and when.

⇒ Why Schedulers are needed:

- Many processes exist at the same time.
- CPU and memory are limited.
- Schedulers help:
  - Select processes
  - Control process flow
  - Improve system performance.







## \* Types of processes (based on Resource usage):

i) CPU bound process: A CPU bound process is a process that spends most of its time performing calculations using the CPU. (high "CPU burst").

### → Characteristics:

- Requires heavy computation
- Uses CPU more than I/O
- Runs continuously with fewer waiting periods.

ii) I/O bound process: An I/O bound process is a process that spends most of its time performing input/output operations.

### → Characteristics:

- frequently waits for devices
- Uses CPU for short time
- More waiting state.

⇒ Why we need good "process mix": ~~we always~~ LTS should always have a good process mix cuz if all processes are I/O bound, the ready queue will always be empty and the STS will have a little to do. if all processes are CPU bound, the I/O waiting queue will be almost empty, devices will go unused, and again the system will be unbalanced. So the system with the best performance will have a good mix / combination of CPU bound and I/O bound processes.



\* Context Switch: Context Switching is the process of saving the state of the current process and loading the state of another process so the CPU can switch execution.

⇒ Why it's needed:

- CPU runs only one process at a time (per core).
- Many processes exist simultaneously.
- OS must switch CPU b/w processes.

→ The "Context" means the current ~~information~~ execution information.

⇒ Behind the scene (how it actually happens):

1. CPU is running a process
2. interrupt or time slice expires
3. OS saves process A data into its PCB
4. OS loads process B data from its PCB
5. CPU starts executing process B



(iii) medium term schedulers: Some operating system such as time sharing systems, may introduce an additional, intermediate level of Scheduling.

- It removes processes from memory temporarily and brings them back later.

→ The process of removing processes from memory temporarily and bringing them back later is known as "swapping".

⇒ when it works:

- Too many processes in RAM
- OS moves some unnecessary process / processes into disk

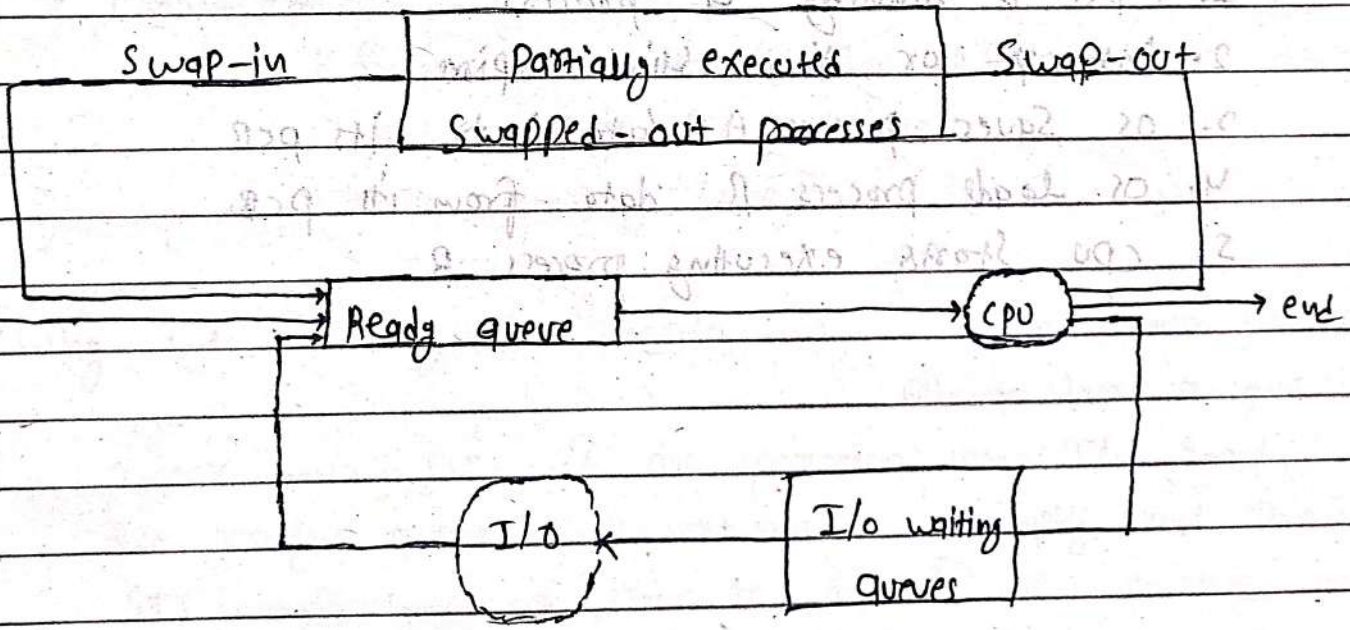


fig: Addition of medium-term scheduler to the queuing diagram



\* Interprocess Communication : interprocess communication (IPC) is the mechanism that allows processes to exchange data and communicate with each other.

The process executing concurrently in operating system may be either "independent processes" or "cooperating processes".

(i) independent process : A process is independent if it cannot affect or be affected by the other processes executing in the system.

→ Any process that does not share data with any other process is independent.

(ii) Cooperating process : A process ~~that~~ is cooperating if it can affect or be affected by the other processes executing in the system.

→ Any process that share data with other processes is a cooperating process.

⇒ The interprocess communication mostly happens b/w "cooperating process".

There are two fundamental models of interprocess communication :

(i) Shared memory

(ii) message passing.



⇒ Zombie process: A zombie process is a process that has finished execution, but its PCB (process entry) still exists in the system.

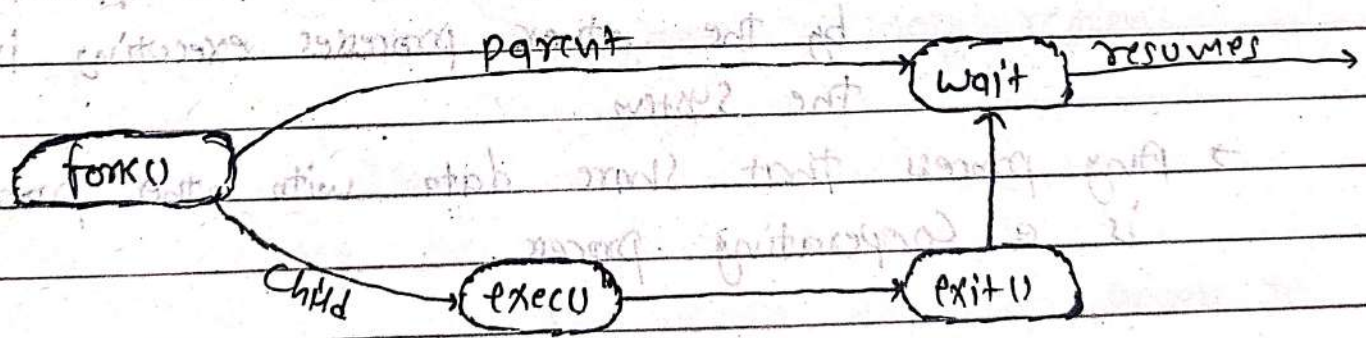
→ why zombie process occurs:

- when a child process finishes.
- But the parent process "has not read its exit status" or "parent got deleted".

⇒ Zombie process !

- Does not use CPU
- Does not run
- only keeps process entry.

To prevent this parent should take child's exit status or parent should get deleted after all child are finished or terminated.





\* Child process: A child process is a new process created by another process, called the parent process.

⇒ When a child processes are created:

- When a program starts another program

- Example:

- You open browser

- Browser opens another tab

- ~~the~~ new tab may run as child process.

OR • Task Division

- Break big task into smaller tasks

- ~~parallel~~ parallel execution:

- parent and child can run together

⇒ Behind the scene: (how child process is created):

→ usually created using:

- `fork()`

Flow:

1. parent process calls `fork()`

2. OS creates new PCB for child containing parent's PID

3. OS allocates new memory space

4. Child process starts execution

5. parent and child run independently.

⇒ Relationship b/w parent and child:

- Child gets copy of:

- Code

- data

- Resources

- Parent may wait for child to finish



2. Sender Sends message using system call:

• `send()`

→ OS stores message temporarily.

3. Receiver requests message:

• `receive()`

→ OS delivers message to receiver

4. Communication Complete, Data safely transferred.

⇒ Blocking and non-blocking message passing:

• Blocking message passing means the sender or receiver waits until the message operation is completed.

→ How it works:

• Blocking send:

• Sender sends message

• `send()` waits until receiver receives message

• Blocking receive:

• Receiver waits until message arrives.

• Non-blocking message passing means the sender or receiver continues execution without waiting.

→ How it works:

• Non blocking send:

• Sender sends message

• Sender continues execution immediately

• Non-blocking receive

• Receiver checks message

• if msg is not available

→ Continues execution



- 5. Synchronization is used to avoid conflicts.
- 6. Memory is removed when processes finish.

⇒ How information being transferred:

- A shared memory is created in RAM
- The sender process writes data in memory.
- The data get updated for all processes immediately coz they are sharing same memory.
- The receive process reads the data and use it.

\* Message Passing: Message passing is an IPC method where processes communicate by sending and receiving message through the operating system.

⇒ Why message passing is needed:

- processes may have separate memory.
- processes can be from distributed computers.

⇒ Where message passing happens:

- messages are handled by the operating system kernel.
- OS manages message transfer b/w processes.

⇒ How message passing works:

1. Sender creates a message

→ message usually contains

- Sender PID
- Receiver PID
- Data.



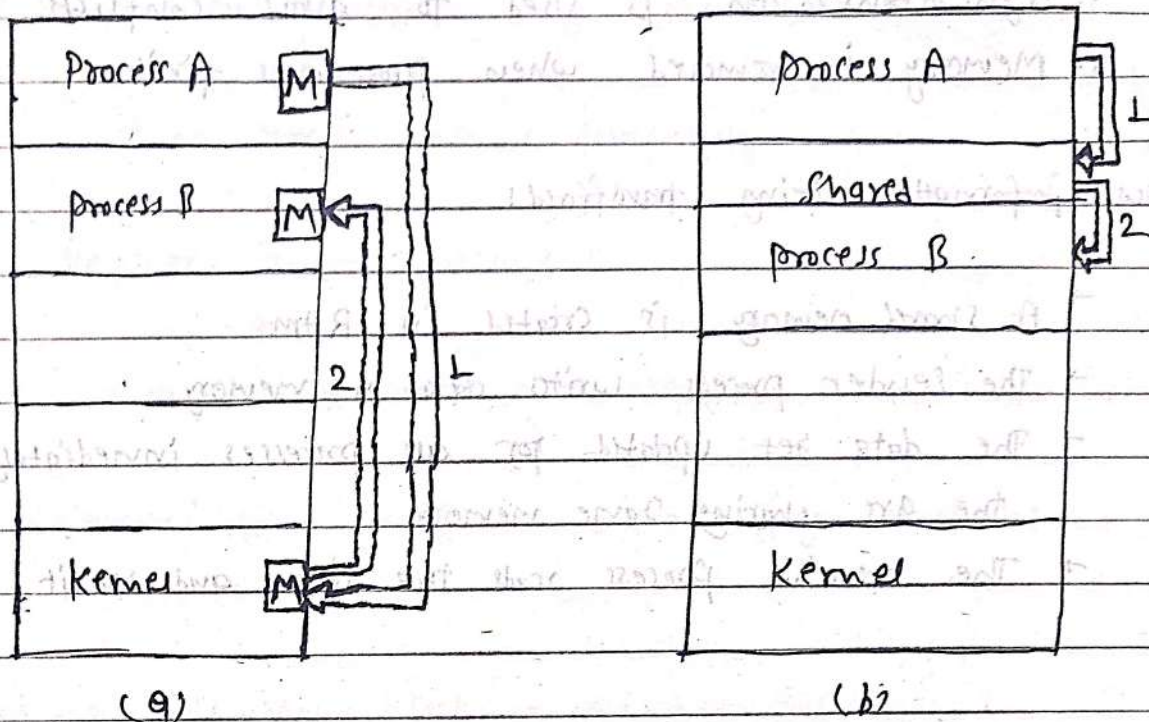


Fig: Communication models: (a) message passing: (b) shared memory.

\* Shared Memory: Shared memory is an ipc method where multiple processes communicate by accessing same memory region.

→ where it is placed:

- created by os in main memory (RAM)
- It is a common memory area, not private to any process.

⇒ How it works:

1. A process request shared memory
2. os create shared memory block in RAM
3. other processes attach to that memory.
4. processes read and write data directly.



⇒ when CPU scheduling happens:

→ CPU scheduling occurs when a process:

- |                |   |                                |              |
|----------------|---|--------------------------------|--------------|
| Non-preemptive | → | • Moves from running → waiting | } Preemptive |
|                | → | • Moves from running → Ready   |              |
|                | → | • Moves from waiting → Ready   |              |
|                | → | • Terminates.                  |              |

\* Scheduling Criteria : Scheduling Criteria are the standards used to evaluate the performance of CPU scheduling algorithms.

→ main Scheduling Criteria:

(1) CPU Utilization :

- percentage of time CPU is busy
- Higher CPU utilization = better performance

Goal: Maximize.

(2) Throughput :

- Number of processes completed per unit time.

Goal: Maximize

(3) Turnaround time :

- total time taken by a process from arrival to completion.

Goal: Minimize

(4) Waiting time :

- total time a process spends waiting in ready queue

Goal: Minimize



## Chapter - 4 : CPU Scheduling.

\* CPU Scheduling : CPU Scheduling is the process of deciding which process gets the CPU and for how long.

⇒ Why CPU Scheduling is needed :

- Many processes are ready to run.
- CPU can execute only one process per core.
- OS must:
  - Share CPU properly
  - Improve system performance
  - Reduce waiting time

Without CPU Scheduling → Chaos Starvation

⇒ Behind the Scenes (how it actually works) :

1. Process enters ready queue
2. Scheduler selects one process
3. Dispatcher gives CPU to selected process
4. Process runs
5. CPU is taken back (interrupt / time slice).
6. Scheduler selects next process.

⇒ Key Goals of CPU Scheduling :

- Maximize CPU Utilization
- Maximize throughput
- ~~Maximize~~ minimize waiting time
- Minimize turnaround time
- Minimize response time
- Fairness.



\* buffering: Buffering is the temporary storage of data/message in memory during communication b/w processes.

⇒ why buffering is needed:

- Sender and receiver may work at different speed.
- buffer stores message until receiver is ready.
- prevents data loss.
- improves communication efficiency.

⇒ where buffer is placed:

- placed in RAM managed by OS.

⇒ Types of buffering:

(i) Zero Capacity buffer (no buffer) \*\*

- buffer size = 0
- Sender must wait until receiver is ready.

(ii) Bounded Capacity Buffer:

- Buffer has limited size
- Sender can send message until buffer is full.

(iii) Unbounded capacity buffer:

- buffer has unlimited size
- Sender never waits
- message stored until receiver reads them.



## Scheduling Algorithms

(1) First Come first serve (FCFS): FCFS is a non-preemptive cpu scheduling Algorithm in which the process that arrives first gets the cpu first.

⇒ How it works:

- Processes are placed in Ready queue
- CPU is allocated in Arrival order
- Once a process gets CPU, it runs till completion or waits for I/O

• Advantages:

- Very Simple
- Easy to implement
- No starvation

• Disadvantages:

- High waiting time
- Convoy Effect (one long process makes others wait)
- ~~Poor~~ Poor response time.

△ It uses simple FIFO queue, no priority, no time slicing.

Example:

| process        | Arrival - time | CPU Burst time |
|----------------|----------------|----------------|
| P <sub>1</sub> | 2              | 6              |
| P <sub>2</sub> | 5              | 2              |
| P <sub>3</sub> | 1              | 8              |
| P <sub>4</sub> | 0              | 3              |
| P <sub>5</sub> | 4              | 4              |

Solution:

Ready queue: [P<sub>4</sub> | P<sub>3</sub> | P<sub>1</sub> | P<sub>5</sub> | P<sub>2</sub>]



: Advantages:

- Simple to implement
- No frequent context switching

: Disadvantages:

- poor Response time
- Long process can block others.

(2) Preemptive Scheduling: in preemptive scheduling, the OS can take CPU away from a running process.

⇒ How it works:

- process is running
- CPU may be taken away if:
  - Time slice expired
  - Higher priority process arrives.
- Context switch occurs

⇒ Examples:

- Round Robin
- Preemptive SJF (Shortest Remaining time first).
- priority scheduling (preemptive)

⇒ Advantages

- Better response time
- good for interactive systems

⇒ Disadvantages

- More Context switching overhead
- More complex.



### (5) Response time :

- Time from process Arrives to first CPU response
- Goal : minimize

⇒ Why these criteria are important :

- they help compare algorithms
- Decide which algorithm is better for:
  - Batch Systems
  - Interactive Systems
  - Real-time Systems.

### \* CPU Scheduling Types :

~~(1) Non-preemptive~~

(1) Non-preemptive Scheduling : in non-preemptive scheduling, Once a process gets the CPU, It keeps the CPU until it finishes or goes into waiting state.

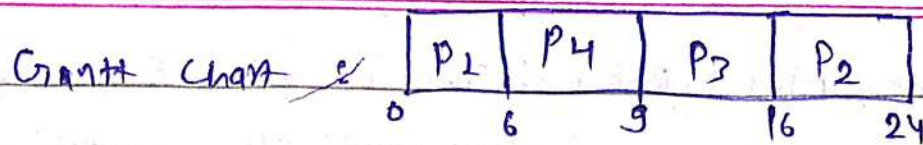
⇒ How it works :

- process starts executing
- CPU is not taken away
- Switch happens only when :
  - process completes or
  - process request I/O

Examples :

- FCFS (first come first serve).
- non-preemptive SJF





Waiting time :  $P_1 = 0 - 0 = 0$

$$P_2 = 16 - 1 = 15$$

$$P_3 = 9 - 2 = 7$$

$$P_4 = 6 - 3 = 3$$

$$\text{Avg WT} = \frac{0 + 15 + 7 + 3}{4} = \frac{25}{4} = 6.25$$

TAT :  $P_1 = 6 - 0 = 6$

$$P_2 = 24 - 1 = 23$$

$$P_3 = 16 - 2 = 14$$

$$P_4 = 9 - 3 = 6$$

~~P~~

$$\text{Avg TAT} = \frac{6 + 23 + 14 + 6}{4} = 11.75$$

$$\text{Throughput} = \frac{5}{24} = 0.20 \text{ process/millisecond}$$



(2) Shortest Job First (SJF): Shortest job first is a CPU scheduling algorithm that selects the process with the smallest CPU burst time first which has arrived.

In one line:

"SJF schedules the process with the shortest time first"

⇒ Type:

• Can be:

- Non-preemptive SJF
- Preemptive SJF (Shortest remaining time = first = SJF).

Non-preemptive SJF:

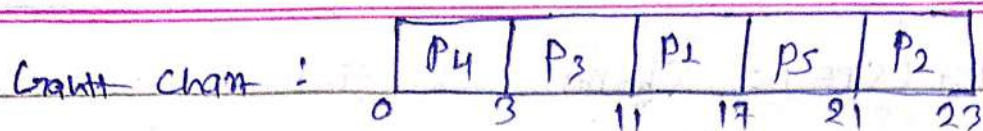
- | Advantages:                                              | Disadvantages:               |
|----------------------------------------------------------|------------------------------|
| • Gives minimum avg waiting time (mathematically proven) | • May cause starvation       |
|                                                          | • Hard to predict burst time |

Example: SJF (non-preemptive):

| processes      | Arrival time | Burst time |
|----------------|--------------|------------|
| P <sub>1</sub> | 0            | 6          |
| P <sub>2</sub> | 1            | 8          |
| P <sub>3</sub> | 2            | 7          |
| P <sub>4</sub> | 3            | 3          |
| P <sub>5</sub> |              |            |

Ready queue → [P<sub>1</sub> | P<sub>2</sub> | P<sub>3</sub> | P<sub>4</sub>]





→ waiting time :  $P_1 = 11 - 2 = 9$

(starting time -  $P_2 = 21 - 5 = 16$

Arrival/dest  $P_3 = 3 - 1 = 2$

(completion time)  $P_4 = 0 - 0 = 0$

$P_5 = 17 - 4 = 3$

Avg. waiting time =  $\frac{9+16+2+0+3}{5} = \frac{30}{5} = 6$  millisen

→ Turnaround time :  $P_1 = 17 - 2 = 15$

~~(Arrival time)~~  $P_2 = 23 - 5 = 18$

(completion time -  $P_3 = 11 - 1 = 10$

Arrival time)  $P_4 = 3 - 0 = 3$

or (waiting time +  $P_5 = 21 - 4 = 17$

Post time)

Avg. Turnaround time =  $\frac{15+18+10+3+17}{5} = \frac{63}{5} = 12.6$

⇒ If the ready queue is empty at any moment CPU will be idle until a process comes in Ready queue

Throughput =  $\frac{5}{23} = 0.21$  process / millisen.



\* Priority Scheduling : Priority Scheduling is a CPU scheduling where the process with the highest priority is executed first.

Exam line :

"Priority scheduling selects the process with the highest priority for execution".

⇒ How priority is Assigned :

- Each process is assigned a priority number.
- Can be :
  - Static (fixed)
  - Dynamic (Changes during execution)



Important :

- Sometimes smaller number = higher priority
- Sometime larger number = higher priority

⇒ Types of priority Scheduling :

→ Non-preemptive priority :

• once process starts → runs until completion

→ Preemptive priority :

- If higher-priority process arrives, current process is preempted.

• Advantages

• Important tasks executed first

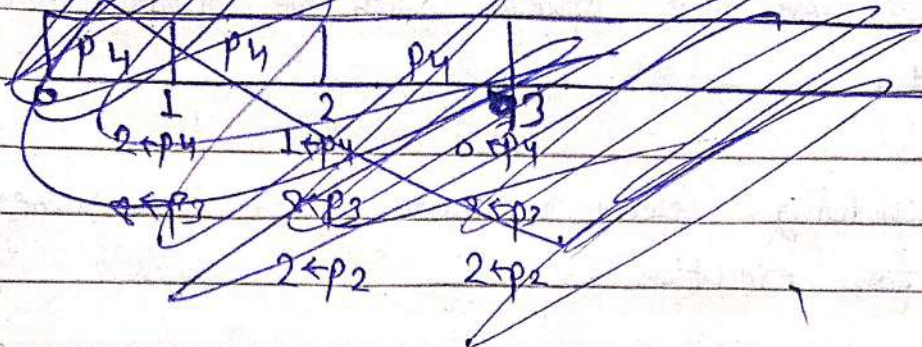
• Good for real-time systems

• Disadvantages

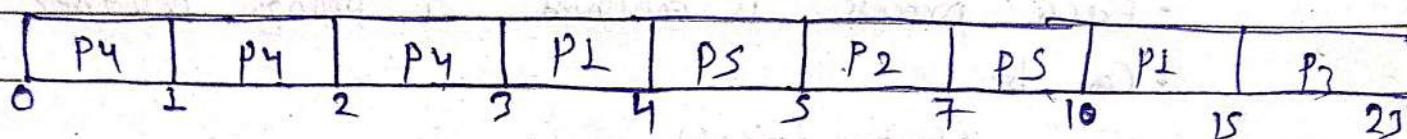
• Starvation (low priority processes may never execute)



~~Gantt Chart:~~



Gantt Chart :



$P4 \rightarrow 3$     $P4 \rightarrow 2$     $P4 \rightarrow 1$     $P4 \rightarrow 0$     $P3 \rightarrow 8$     $P3 \rightarrow 8$     $P3 \rightarrow 8$     $P3 \rightarrow 8$     $P3 \rightarrow 8$     $P3 \rightarrow 0$   
 $P3 \rightarrow 8$     $P3 \rightarrow 8$     $P3 \rightarrow 8$     $P1 \rightarrow 5$     $P1 \rightarrow 5$     $P2 \rightarrow 5$     $P1 \rightarrow 5$     $P1 \rightarrow 0$   
 $P1 \rightarrow 6$     $P1 \rightarrow 6$     $P5 \rightarrow 4$     $P5 \rightarrow 3$     $P5 \rightarrow 3$     $P5 \rightarrow 0$   
 $P2 \rightarrow 2$     $P2 \rightarrow 0$

$$\begin{aligned}
 \text{Waiting time} = & \left. \begin{aligned} P1 &= (3-2) + (10-4) = 7 \\ P3 &= 5-5 = 0 \\ P3 &= 15-1 = 14 \\ P4 &= 0-0 = 0 \\ P5 &= (4-4) + (7-5) = 2 \end{aligned} \right\} \rightarrow \frac{23}{5} = 4.6
 \end{aligned}$$

TAT = (Waiting time + Dist time)

$$\begin{aligned}
 P1 &= 7 + 6 = 13 \\
 P2 &= 0 + 2 = 2 \\
 P3 &= 14 + 8 = 22 \\
 P4 &= 0 + 3 = 3 \\
 P5 &= 2 + 4 = 6
 \end{aligned}
 \quad \rightarrow \quad 48/5 = 9.6$$

$$\text{Throughput} = \frac{5}{23} = 0.21 \text{ process/millisecond}$$



\* Shortest Remaining time First (SRTF) : This is also known as preemptive STF where the process with the Smallest Remaining CPU burst time is selected for execution from the Ready queue.

→ How it works!

1. At any time, check all ready processes.
2. select the one with smallest Remaining time
3. if a new process arrives with smaller Remaining time :

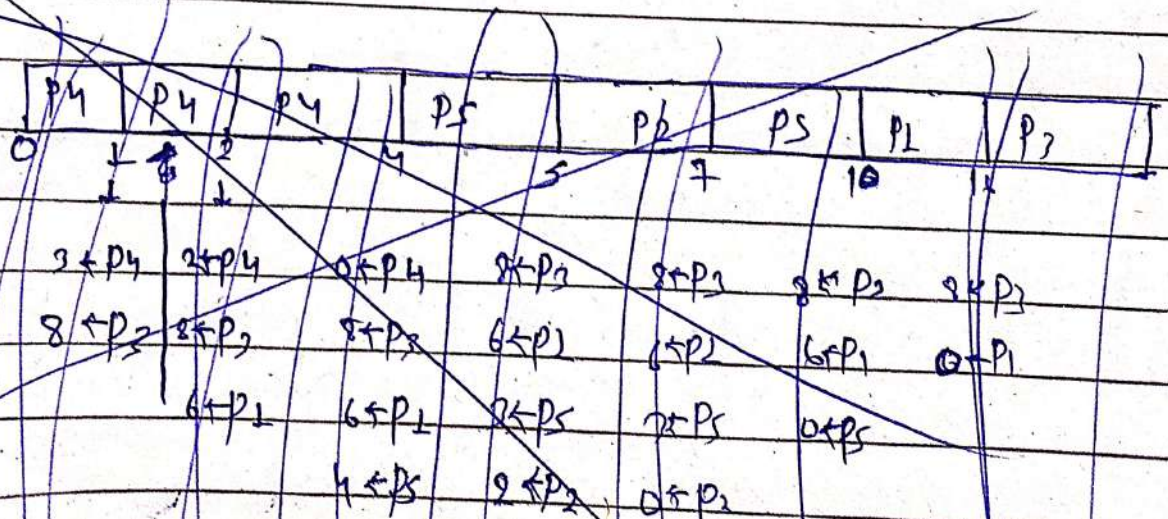
- stop current process
- Perform Context Switch
- Run new Shorter process.

~~Example~~

Example :

| process | Arrival time | Post time |
|---------|--------------|-----------|
| P1      | 2            | 6         |
| P2      | 5            | 2         |
| P3      | 1            | 8         |
| P4      | 0            | 3         |
| P5      | 4            | 4         |

Solution : Gantt chart :





and  
~~the~~ Verifying

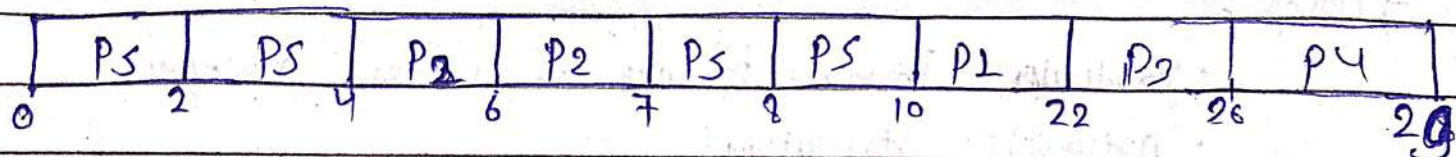
Segment 1 = 1001001110010011  
Segment 2 1001100001001101  
Checksum 1101010000011110

---

0



Preemptive :



$P5 \rightarrow 7$   $P5 \rightarrow 5$   $P5 \rightarrow 3$   $P5 \rightarrow 3$   $P5 \rightarrow 3$   $P5 \rightarrow 2$   $P5 \rightarrow 0$   $P4 \rightarrow 3$   $P4 \rightarrow 3$   
 $P4 \rightarrow 3$   $P4 \rightarrow 3$   $P4 \rightarrow 3$   $P4 \rightarrow 3$   $P4 \rightarrow 3$   $P4 \rightarrow 2$   $P1 \rightarrow 0$   $P3 \rightarrow 0$   
 $P2 \rightarrow 3$   $P2 \rightarrow 1$   $P2 \rightarrow 0$   $P1 \rightarrow 10$   $P1 \rightarrow 12$   $P3 \rightarrow 4$   
 $P1 \rightarrow 12$   $P1 \rightarrow 12$   $P3 \rightarrow 4$   $P3 \rightarrow 4$

$$\begin{aligned}
 \text{WIT} : & \quad P1 = 10 - 6 = 4 \\
 & \quad P2 = 4 - 4 = 0 \\
 & \quad P3 = 22 - 8 = 14 \\
 & \quad P4 = 26 - 2 = 24 \\
 & \quad P5 = (0 - 0) + (7 - 4) = 3
 \end{aligned}
 \quad \left. \begin{array}{l} \\ \\ \\ \\ \end{array} \right\} \frac{45}{5} = 9$$

$$\begin{aligned}
 \text{TAT} = & \quad P1 = 4 + 12 = 16 \\
 & \quad P2 = 0 + 3 = 3 \\
 & \quad P3 = 14 + 4 = 18 \\
 & \quad P4 = 24 + 3 = 27 \\
 & \quad P5 = 3 + 7 = 10
 \end{aligned}
 \quad \left. \begin{array}{l} \\ \\ \\ \\ \end{array} \right\} \frac{74}{5} = 14.8$$

Throughput :  $\frac{5}{29} = 0.17 \text{ p/ms.}$



⇒ Solution to starvation :

→ Aging :

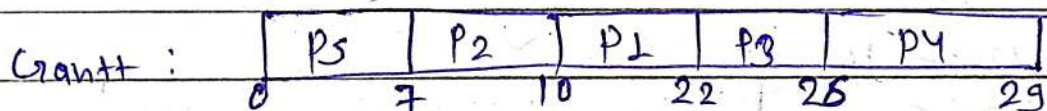
- gradually increase priority of waiting processes
- prevents starvation.

Example : ~~Process~~

| process        | Arrival time | Burst time | priority |
|----------------|--------------|------------|----------|
| P <sub>1</sub> | 6            | 12         | 3        |
| P <sub>2</sub> | 4            | 3          | 1        |
| P <sub>3</sub> | 8            | 4          | 3        |
| P <sub>4</sub> | 2            | 3          | 4        |
| P <sub>5</sub> | 0            | 7          | 2        |

→ Smaller number implies higher priority

Solution : (preemptive) :



W.T!  $P_1 = 10 - 6 = 4$

$P_2 = 7 - 4 = 3$

$P_3 = 22 - 8 = 14$

$P_4 = 26 - 2 = 24$

$P_5 = 0 - 0 = 0$

$45/5 = 9$

TAT : (WT + DT)

Through put =  $\frac{5}{29} = 0.17 \text{ P/ms}$

$P_1 = 4 + 12 = 16$

$P_2 = 3 + 3 = 6$

$P_3 = 14 + 4 = 18$

$P_4 = 24 + 3 = 27$

$P_5 = 0 + 7 = 7$

$74/5 = 14.8$